# Kernel Methods and the Mercer Theorem

### Machine Learning — Week 8

> **The one-line story:** when a straight line cannot separate your data, don't bend the line — *lift the data* into a space where a straight line works. Kernels let you do the lifting **without ever building the lifted space**.

---

## Index


- Why Kernels Exist — *Flatland has a problem: some data refuses to line up politely.*
- The Kernel Trick Inside the SVM — *Spoiler: the SVM only ever asks "how alike are these two points?"*
- What Exactly Is a Kernel — *A similarity-meter that secretly lives in a higher dimension.*
- Positive Definite Kernels — *The dress code: which similarity-meters are allowed in.*
- The Mercer Theorem — *The guarantee that the secret lifted space really exists.*
- Proof Toolkit Eigen Decomposition — *Eigenvectors: the vectors that refuse to change direction.*
- Proof of the Mercer Theorem — *Taking a matrix apart, then reassembling it as a dot product.*
- Putting Kernels to Work — *The recipe card.*
- A Zoo of Kernels — *Polynomial, Gaussian, Laplacian, Sigmoid — meet the animals.*
- Kernel Linear Regression — *Ridge regression, but wearing a kernel costume.*
- Practice Questions — *Five puzzles to make it stick.*


---

## Why Kernels Exist

We are given a data set

$$\mathrm{X} = \{x_1, x_2, \ldots, x_n\}, \qquad x_i \in \mathrm{R}^d$$

where each data point lives in a $d$-dimensional space. In many classification problems the two classes are **not linearly separable** in $\mathrm{R}^d$ — no straight line (or hyperplane) can split them.

**The idea:** map every data point to a new, higher-dimensional space $\mathrm{R}^m$ with $m > d$:

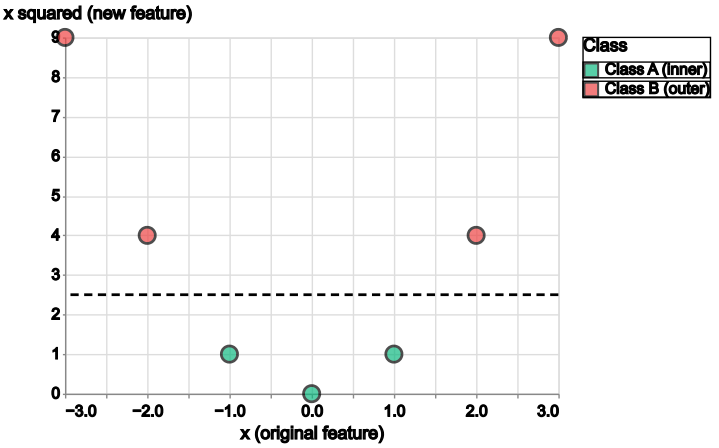
$$\phi : \mathrm{X} \to \mathrm{R}^m, \qquad m > d$$

In that bigger space the two classes **become linearly separable**.

### A tiny picture of "lifting"

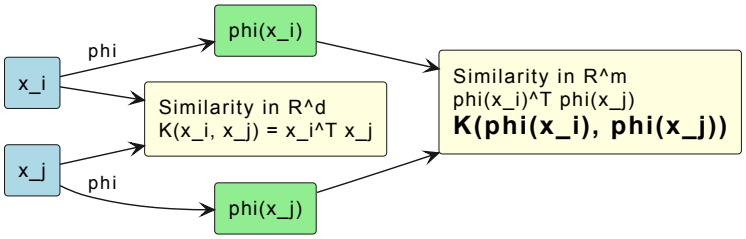
Take 1-D points: the inner class sits near $0$, the outer class far from $0$. No single threshold on the line separates them. Add one new coordinate, $x \mapsto (x, x^2)$, and a horizontal line does the job.

The dashed line (separating hyperplane) lives in the lifted space.



### The catch — and the way out

Do we actually need the explicit mapping $\phi$? **No.** All we need is a way to *compare* two data points in the higher-dimensional space, i.e. a **similarity function**.




| Symbol | Meaning |
|---|---|
| $\mathrm{X}$ | the data set $\{x_1, \dots, x_n\}$, with $x_i \in \mathrm{R}^d$ |
| $\phi$ | feature map $\mathrm{X} \to \mathrm{R}^m$, $m > d$ |
| $K$ | kernel $K : \mathrm{X} \times \mathrm{X} \to \mathrm{R}$, returns a scalar similarity |

---

## The Kernel Trick Inside the SVM

Recall the SVM optimisation problem. The lecture writes it schematically as

$$\min_{w,b} \; w^\top w \; - \; \sum_{i=1}^{n} \alpha_i \, y_i (x_i^\top w - b) \; \cdots$$

- $w, b$ are the **primal** variables.
- $\alpha_i$ (one per data point, $i = 1, \dots, n$) are the **dual** variables (Lagrange multipliers).

> **Note for Review — Pro-Tip:** the schematic form above drops constants. The complete Lagrangian is $\mathrm{L}(w, b, \alpha) = \frac{1}{2} w^\top w - \sum_i \alpha_i [y_i (x_i^\top w - b) - 1]$ with $\alpha_i \geq 0$. The $\frac{1}{2}$ and the "$-1$" are what produce the $\frac{1}{2}$ and the $\sum_i \alpha_i$ in the dual below.

### Applying the KKT conditions

Setting the derivatives of $\mathrm{L}$ to zero:

$$\frac{\partial \mathrm{L}}{\partial w} = w - \sum_{i=1}^{n} \alpha_i y_i x_i = 0 \quad \Rightarrow \quad w = \sum_{i=1}^{n} \alpha_i y_i x_i$$

$$\frac{\partial \mathrm{L}}{\partial b} = \sum_{i=1}^{n} \alpha_i y_i = 0$$

Substitute $w$ back. Each of the two quadratic pieces becomes a double sum:

$$w^\top w = \sum_i \sum_j \alpha_i \alpha_j y_i y_j \, x_i^\top x_j, \qquad \sum_i \alpha_i y_i \, x_i^\top w = \sum_i \sum_j \alpha_i \alpha_j y_i y_j \, x_i^\top x_j$$

so $\frac{1}{2} S - S + b \sum_i \alpha_i y_i + \sum_i \alpha_i$ with $S$ the double sum, and the $b$-term vanishes because $\sum_i \alpha_i y_i = 0$. What remains is the **dual cost function**:

$$\sum_i \alpha_i \; - \; \frac{1}{2} \sum_i \sum_j \alpha_i \, \alpha_j \, y_i \, y_j \, \langle x_i, x_j \rangle$$

### The key observation

In the dual, the data points appear **only through the inner product** $\langle x_i, x_j \rangle$ — i.e. only through a *similarity* between pairs of points.

So to work in the lifted space we just replace that similarity by the similarity of the lifted points:

$$\langle x_i, x_j \rangle \quad \longrightarrow \quad K(\phi(x_i), \phi(x_j)) = \phi(x_i)^\top \phi(x_j)$$

No explicit $\phi$ is ever needed if we can compute $K$ directly from the original points. That is the **kernel trick**.

## What Exactly Is a Kernel

A kernel $K$ is simply a **function that takes two data points as input and returns a scalar (their similarity)**:

$$K : \mathrm{X} \times \mathrm{X} \to \mathrm{R}$$

### Example 1 — Polynomial kernel

$$K(x_i, x_j) = (\langle x_i, x_j \rangle + c)^2$$

The power $2$ can be any other power $(4, 6, \ldots)$. The same formula gives similarity in either space:

- original space: plug in $x_i, x_j$;
- higher-dimensional space: plug in $\phi(x_i)$, $\phi(x_j)$.

The remarkable fact (seen in the previous class) is that this can be written as

$$K(x_i, x_j) = \phi(x_i)^\top \phi(x_j)$$

**Worked check** ($d = 2$, power $2$). Let $x = (x_1, x_2)$, $x' = (x_1', x_2')$ and put $a = x_1 x_1'$, $b = x_2 x_2'$. Then

$$(x^\top x' + c)^2 = (a + b + c)^2 = a^2 + b^2 + c^2 + 2ab + 2ac + 2bc$$

$$= x_1^2 x_1'^2 + x_2^2 x_2'^2 + 2\, x_1 x_2\, x_1' x_2' + 2c\, x_1 x_1' + 2c\, x_2 x_2' + c^2$$

This is exactly $\phi(x)^\top \phi(x')$ for

$$\phi(x) = (x_1^2,\ x_2^2,\ \sqrt{2}\, x_1 x_2,\ \sqrt{2c}\, x_1,\ \sqrt{2c}\, x_2,\ c)^\top,$$

so a $2$-D input is lifted to $m = 6$ dimensions — yet computing the kernel only costs one dot product in $\mathrm{R}^2$ plus a square.

### Example 2 — Gaussian kernel

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|_2^2}{2\sigma^2}\right)$$

Another way of measuring similarity: close points give a value near $1$, distant points a value near $0$.

### Why is it called a "trick"?

To get the similarity of two points **in the higher-dimensional space**, we only need the **original** data points and the kernel formula. We never need to know — or compute — the mapping $\phi$.

The open question: *why does such a $\phi$ exist at all for a given kernel formula?* The next sections answer it with the **Mercer theorem**.

## Positive Definite Kernels

Let the kernel be $K : \mathrm{X}^2 \to \mathrm{R}$, where $\mathrm{X}^2$ is shorthand for $\mathrm{X} \times \mathrm{X}$. Given $\mathrm{X} = \{x_1, \ldots, x_n\}$, compute the similarity between **all** pairs $(x_i, x_j)$, $i, j \in \{1, \ldots, n\}$, and stack them into the **kernel matrix** (a.k.a. Gram matrix):

$$K = \begin{bmatrix} K(x_1, x_1) & \cdots & K(x_1, x_n) \\ \vdots & \ddots & \vdots \\ K(x_n, x_1) & \cdots & K(x_n, x_n) \end{bmatrix} \in \mathrm{R}^{n \times n}$$

(Row $1$ holds the similarity of $x_1$ with every point; the last entry of the last row is the similarity of $x_n$ with itself. There are $n$ points and every pair is compared, hence $n \times n$.)

### Symmetric kernel

Similarity of $x_i$ to $x_j$ should equal similarity of $x_j$ to $x_i$:

$$K(x_i, x_j) = K(x_j, x_i) \quad \text{for all pairs of data points}$$

The polynomial and Gaussian kernels above are both symmetric.

### Positive definite and positive semi-definite

| Type | Condition on the kernel matrix |
|---|---|
| Positive definite (PD) | $c^\top K c > 0$ for every non-zero $c \in \mathrm{R}^n$ |
| Positive semi-definite (PSD) | $c^\top K c \geq 0$ for every $c \in \mathrm{R}^n$ |

Here $c$ is any $n \times 1$ vector. Since the kernel is represented by a matrix, we use "PSD kernel" and "PSD matrix" interchangeably.

**Why do we care?** For a symmetric PSD kernel matrix we can always **find the corresponding $\phi$** — this is exactly what the Mercer theorem says.

### Worked check of PSD

Take $K = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ and any $c = (c_1, c_2)^\top$:

$$c^\top K c = \begin{bmatrix} c_1 & c_2 \end{bmatrix} \begin{bmatrix} 2c_1 + c_2 \\ c_1 + 2c_2 \end{bmatrix} = 2c_1^2 + 2c_1 c_2 + 2c_2^2 = c_1^2 + c_2^2 + (c_1 + c_2)^2 \;\geq\; 0$$

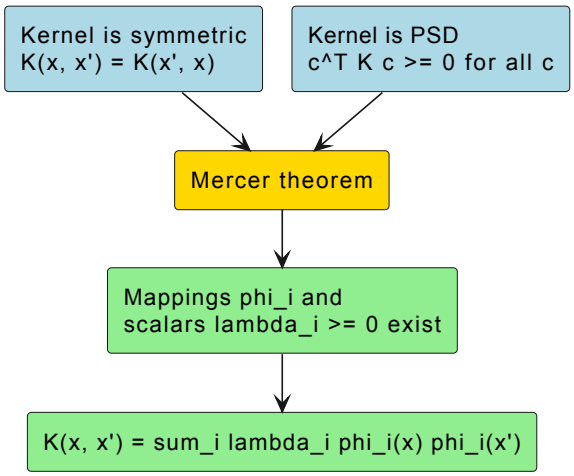
A sum of squares — so $K$ is PSD. (We'll reuse this matrix later.)

---

## The Mercer Theorem

> **Mercer's theorem.** Let $K : \mathrm{X}^2 \to \mathrm{R}$ be a **symmetric** kernel which is **positive semi-definite**. Then there exist mappings $\phi_i$ and scalars $\lambda_i \geq 0$ such that, for any two data points $x$ and $x'$,
>
> $K(x, x') = \sum_i \lambda_i \, \phi_i(x) \, \phi_i(x')$

In words: **symmetric + PSD ⇒ the kernel is secretly a dot product in some feature space.** The proof will even show *how to compute* the map.



# Proof Toolkit Eigen Decomposition

For the proof we work with the kernel matrix with the two properties we are given:

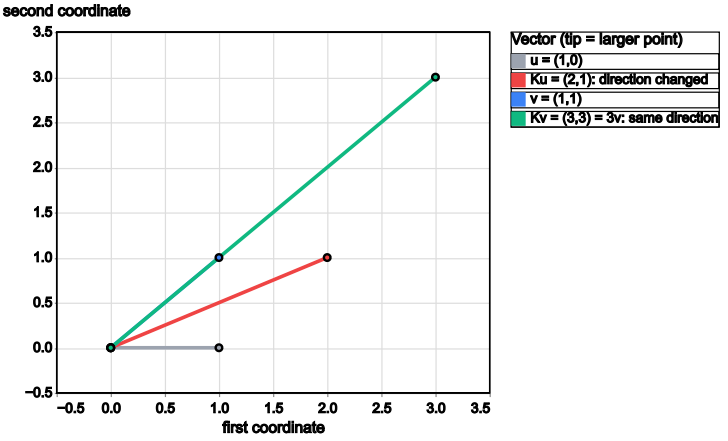
$$K \in \mathrm{R}^{n \times n}, \qquad K^{\top} = K, \qquad c^{\top} K c \geq 0 \;\; \forall\, c \in \mathrm{R}^{n}$$

## Eigenvalues and eigenvectors (quick refresher)

A vector $v$ is an **eigenvector** of $K$ with **eigenvalue** $\lambda$ if

$$Kv = \lambda v$$

Geometrically: for most vectors, multiplying by $K$ turns them to point in a new direction. Eigenvectors are the special ones where $Kv$ points along the **same direction** as $v$ — only stretched or shrunk (by the scalar $\lambda$).




Concrete picture with $K = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$:

- $u = (1, 0)$ gives $Ku = (2, 1)$ — **direction changes**, so $u$ is *not* an eigenvector.
- $v = (1, 1)$ gives $Kv = (3, 3) = 3v$ — **same direction**, so $v$ is an eigenvector with $\lambda = 3$.

An n\times n matrix has **at most n distinct eigenvalues**, with corresponding eigenvectors:

$$Kv_1 = \lambda_1 v_1, \quad \ldots, \quad Kv_n = \lambda_n v_n$$

## Clubbing all $n$ equations into one

Place the eigenvectors as columns of one matrix and the eigenvalues on a diagonal:

$$K \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix} = \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix} \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_n \end{bmatrix}$$

(Multiply out both sides: the $k$-th column of each side is $Kv_k$ and $\lambda_k v_k$, so you recover all $n$ equations.) In short,

$$KV = VD, \qquad V = \begin{bmatrix} v_1 & \cdots & v_n \end{bmatrix}, \qquad D = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$$

Eigenvectors belonging to distinct eigenvalues are linearly independent, so $V$ is invertible. Multiply on the right by $V^{-1}$:

$$KVV^{-1} = VDV^{-1} \quad \Rightarrow \quad K = VDV^{-1}$$

This is the **eigenvalue–eigenvector decomposition** of $K$.

## Why can we order the eigenvalues, and why are they $\geq 0$?

- For a general matrix, eigenvalues can be complex, so they cannot be ordered.
- For a **symmetric** matrix, eigenvalues are always **real**.
- For a **PSD** matrix they are moreover **non-negative**.

So we may order them: $\lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$.

**Proof that $\lambda \geq 0$.** Start from $Kv = \lambda v$ and multiply by $v^{\top}$ on the left:

$$v^{\top} K v = \lambda\, v^{\top} v$$

The left side is $\geq 0$ by the PSD condition (take $c = v$). And $v^{\top} v = \|v\|^2 > 0$ for an eigenvector ($v \neq 0$). Therefore

$$\lambda\, v^{\top} v \geq 0 \quad \Longrightarrow \quad \boxed{\lambda \geq 0}$$

### Eigenvectors of a symmetric matrix are orthogonal

Let $K^\top = K$, take two **distinct** eigenvalues $\lambda_i \neq \lambda_j$ with eigenvectors $v_i, v_j$:

$$Kv_i = \lambda_i v_i, \qquad Kv_j = \lambda_j v_j$$

Take the inner product of the first equation with $v_j$ (the inner product is symmetric, and $\lambda_i$ is a scalar so it pulls out):

$$\langle v_j, Kv_i \rangle = \langle v_j, \lambda_i v_i \rangle \;\Rightarrow\; \langle Kv_i, v_j \rangle = \lambda_i \langle v_i, v_j \rangle$$

Apply the definition of the inner product, $\langle a, b \rangle = a^\top b$:

$$(Kv_i)^\top v_j = \lambda_i\, v_i^\top v_j \;\Rightarrow\; v_i^\top K^\top v_j = \lambda_i\, v_i^\top v_j$$

Use $K^\top = K$, then use $Kv_j = \lambda_j v_j$:

$$v_i^\top K v_j = \lambda_i\, v_i^\top v_j \;\Rightarrow\; v_i^\top \lambda_j v_j = \lambda_i\, v_i^\top v_j \;\Rightarrow\; \lambda_j\, v_i^\top v_j = \lambda_i\, v_i^\top v_j$$

Bring everything to one side:

$$(\lambda_i - \lambda_j)\, v_i^\top v_j = 0$$

Because $\lambda_i \neq \lambda_j$, the bracket is non-zero, hence

$$\boxed{v_i^\top v_j = 0 \quad (i \neq j)}$$

Eigenvectors can be scaled to **unit length**, so

$$\langle v_i, v_j \rangle = \begin{cases} 0 & i \neq j \\ 1 & i = j \end{cases}$$

> **Note for Review — Pro-Tip:** this often appears on exams as "why are the eigenvectors of a symmetric matrix orthogonal?". The whole proof is the one-liner $(\lambda_i - \lambda_j)\, v_i^\top v_j = 0$. If an eigenvalue is repeated you simply *choose* an orthonormal basis inside that eigenspace (spectral theorem), so $V$ is still orthonormal.

---

## Proof of the Mercer Theorem

### Step 1 — $V$ is orthonormal

The columns of $V = [v_1 \;\cdots\; v_n]$ are mutually orthogonal and of unit length, so $V$ is an **orthonormal matrix**:

$$V^\top V = I \quad\Rightarrow\quad V^{-1} = V^\top$$

Hence the decomposition becomes

$$K = V D V^{-1} = V D V^\top$$

### Step 2 — Split $D$ into two square roots

$D$ is diagonal, so $D^{1/2}$ just takes the square root of each diagonal entry (real, because all $\lambda_l \geq 0$):

$$D^{1/2} = \begin{bmatrix} \sqrt{\lambda_1} & & 0 \\ & \ddots & \\ 0 & & \sqrt{\lambda_n} \end{bmatrix}, \qquad D = D^{1/2} D^{1/2}$$

Therefore

$$K = V D^{1/2} D^{1/2} V^\top$$

### Step 3 — Rewrite as "something transposed times something"

Use the identity $AB = (B^\top A^\top)^\top$ with $A = V D^{1/2}$ and $B = D^{1/2} V^\top$. Since $D^{1/2}$ is diagonal, $(D^{1/2})^\top = D^{1/2}$, so

$$A = V D^{1/2} = (D^{1/2} V^\top)^\top$$

and

$$K = (D^{1/2} V^\top)^\top (D^{1/2} V^\top)$$

## Step 4 — Read off one entry

The $(i,j)$ entry of a product is *row $i$ of the left factor* times *column $j$ of the right factor*. Row $i$ of $(D^{1/2}V^\top)^\top$ is the transpose of column $i$ of $D^{1/2}V^\top$, so

$$K_{ij} = ((D^{1/2}V^\top)_{:i})^\top (D^{1/2}V^\top)_{:j}$$

Define the feature vector of data point $x_i$ as the $i$-th column of $D^{1/2}V^\top$:

$$\theta(x_i) := (D^{1/2}V^\top)_{:i}$$

Then

$$\boxed{K_{ij} = K(x_i, x_j) = \theta(x_i)^\top \theta(x_j)}$$

Stacking the vectors $\theta(x_i)^\top$ as the **rows** of a matrix $\Theta$ gives $\Theta = V D^{1/2}$ and

$$K = \Theta\,\Theta^\top$$

## Step 5 — The same result from the outer-product form (entry by entry)

Multiply the three matrices $V$, $D$, $V^\top$ explicitly:

$$K = VDV^\top = [\,v_1 \;\cdots\; v_n\,] \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_n \end{bmatrix} \begin{bmatrix} v_1^\top \\ \vdots \\ v_n^\top \end{bmatrix}$$

First, $VD = [\,\lambda_1 v_1 \;\cdots\; \lambda_n v_n\,]$ (each column is scaled by its eigenvalue). Multiplying by the stacked rows $v_l^\top$ adds up one rank-one matrix per eigenvector:

$$K = \sum_{l=1}^{n} \lambda_l\, v_l\, v_l^\top$$

Here $v_l$ is the whole $l$-th eigenvector and $v_l(i)$ denotes its $i$-th entry. The $(i,j)$ entry of $v_l v_l^\top$ is $v_l(i)\,v_l(j)$, so

$$[K]_{ij} = K(x_i, x_j) = \Big(\sum_{l=1}^{n} \lambda_l\, v_l v_l^\top\Big)_{ij} = \sum_{l=1}^{n} \lambda_l\, v_l(i)\, v_l(j)$$

Split each $\lambda_l$ into $\lambda_l^{1/2}\lambda_l^{1/2}$ (legal because $\lambda_l \geq 0$):

$$[K]_{ij} = \sum_{l=1}^{n} \lambda_l^{1/2} v_l(i)\; \lambda_l^{1/2} v_l(j)$$

A sum of products of matching entries is the **inner product of two vectors**:

$$[K]_{ij} = [\lambda_1^{1/2}v_1(i) \quad \lambda_2^{1/2}v_2(i) \quad \cdots \quad \lambda_n^{1/2}v_n(i)] \begin{bmatrix} \lambda_1^{1/2}v_1(j) \\ \lambda_2^{1/2}v_2(j) \\ \vdots \\ \lambda_n^{1/2}v_n(j) \end{bmatrix} = \theta(x_i)^\top \theta(x_j)$$

with

$$\theta(x_i) = \big(\sqrt{\lambda_1}\,v_1(i),\; \sqrt{\lambda_2}\,v_2(i),\; \ldots,\; \sqrt{\lambda_n}\,v_n(i)\big)^\top$$

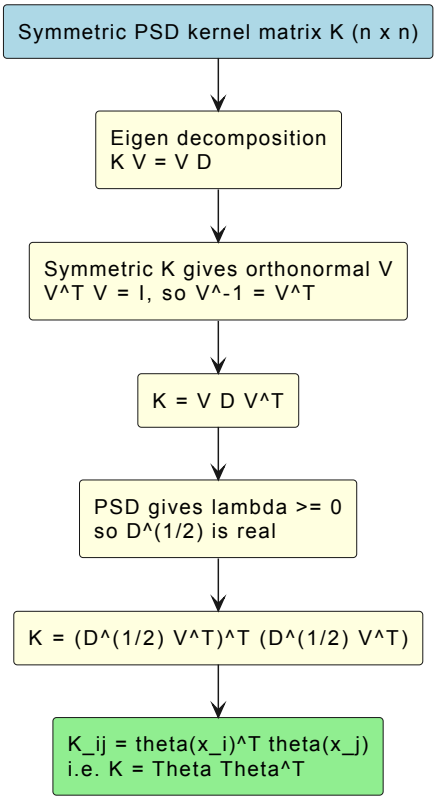
### Step 6 — Match with the statement of the theorem

Define $\phi_l := v_l$, i.e. $\phi_l(x_i) = v_l(i)$. Then Step 5 reads

$$K(x_i, x_j) = \sum_{l=1}^{n} \lambda_l \, \phi_l(x_i) \, \phi_l(x_j), \qquad \lambda_l \geq 0$$

which is exactly the Mercer form. The explicit feature map is $\theta(x) = (\sqrt{\lambda_l}\, \phi_l(x))_l$. ■

(Notation reminder: in the derivation the eigenvectors play the role of the $\phi_l$ in the theorem, while the final lifted vector is called $\theta$.)




### Fully worked numerical example

Take $K = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ (symmetric, PSD as checked earlier).

**Eigenvalues.** Solve $\det(K - \lambda I) = (2-\lambda)^2 - 1 = 0 \Rightarrow \lambda = 1$ or $3$. Ordered: $\lambda_1 = 1,\ \lambda_2 = 3$.

**Eigenvectors (unit length).** $\lambda_1 = 1$: $(K - I)v = 0 \Rightarrow v_1 = \frac{1}{\sqrt{2}}(1, -1)^\top$. $\lambda_2 = 3$: $v_2 = \frac{1}{\sqrt{2}}(1, 1)^\top$. Check $v_1^\top v_2 = \frac{1}{2}(1 - 1) = 0$ ✓.

$$V = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}, \qquad D = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$$

**Verify $K = VDV^\top$ (matrix multiplication shown).**

$$VD = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \cdot 1 & 1 \cdot 3 \\ -1 \cdot 1 & 1 \cdot 3 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 3 \\ -1 & 3 \end{bmatrix}$$

$$VDV^\top = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 3 \\ -1 & 3 \end{bmatrix} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1+3 & -1+3 \\ -1+3 & 1+3 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \ \checkmark$$

**Feature vectors.** Using $\theta(x_i) = (\sqrt{\lambda_1}\, v_1(i), \sqrt{\lambda_2}\, v_2(i))^\top$:

$$\theta(x_1) = \left(1 \cdot \tfrac{1}{\sqrt{2}},\ \sqrt{3} \cdot \tfrac{1}{\sqrt{2}}\right)^\top, \qquad \theta(x_2) = \left(1 \cdot \tfrac{-1}{\sqrt{2}},\ \sqrt{3} \cdot \tfrac{1}{\sqrt{2}}\right)^\top$$

**Check the dot products.**

$$\theta(x_1)^\top \theta(x_1) = \tfrac{1}{2} + \tfrac{3}{2} = 2 = K_{11}, \qquad \theta(x_1)^\top \theta(x_2) = -\tfrac{1}{2} + \tfrac{3}{2} = 1 = K_{12}, \qquad \theta(x_2)^\top \theta(x_2) = \tfrac{1}{2} + \tfrac{3}{2} = 2 = K_{22}$$

All entries of $K$ are recovered as dot products of explicit feature vectors ✓.

### Why non-negative eigenvalues matter

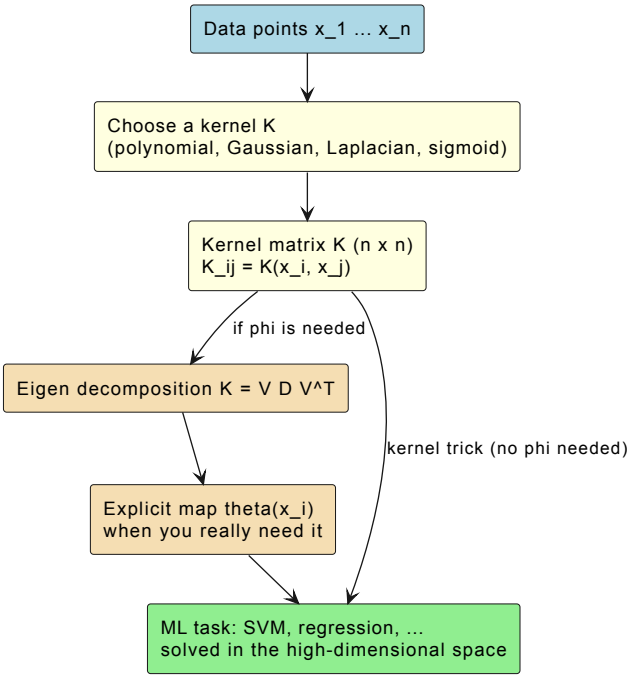
If some $\lambda_l$ were negative, then $\sqrt{\lambda_l}$ would be imaginary: there would be **no real vector** $\theta(x)$ and the "similarity" could even turn negative for a point with itself ($K_{ii} = \|\theta(x_i)\|^2$ must be $\geq 0$). The PSD condition $c^\top K c \geq 0$ is precisely what guarantees $\lambda_l \geq 0$, hence real feature vectors and non-negative self-similarity.

## Putting Kernels to Work

Recipe:

1. Start with your data points $\mathrm{X} = \{x_1, \ldots, x_n\}$.
2. Choose a kernel $K$ (polynomial, Gaussian, Laplacian, sigmoid, …).
3. Build the kernel matrix $K \in \mathrm{R}^{n \times n}$ using that formula.
4. Compute its eigenvalue–eigenvector decomposition.
5. Build the mapping $\theta$ (the "$\phi$") from it — this lifts your points to a higher-dimensional space.
6. Solve any machine learning task there.

There are two routes: if the algorithm only needs similarities (SVM dual, kernel regression below) you go **directly** from $K$ to the task and skip the eigen-step; if you explicitly need the lifted points, you go via the decomposition.




## A Zoo of Kernels

| Kernel | Formula | Parameters |
|---|---|---|
| Polynomial | $K(x, x') = (x^\top x' + c)^d$ | constant $c$, degree $d$ |
| Gaussian | $K(x, x') = \exp(-\frac{\lVert x - x' \rVert_2^2}{2\sigma^2})$ | width $\sigma$ |
| Laplacian | $K(x, x') = \exp(-\frac{\lVert x - x' \rVert_1^2}{2\sigma^2})$ | width $\sigma$ |
| Sigmoid | $K(x, x') = \tanh(a\, x^\top x' + b)$ | $a, b > 0$ |

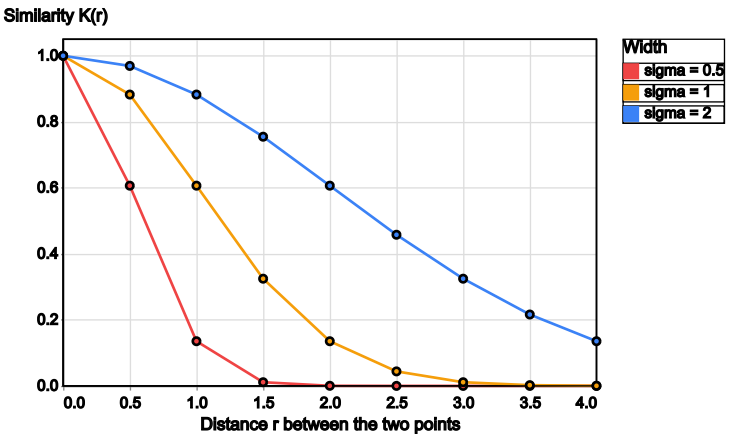
All of these can map data to high-dimensional spaces; with a kernel in hand you get similarity scores in the lifted space **without knowing** $\phi$. If you do need $\phi$, build the kernel matrix and run the eigen-decomposition from the proof above.

### Radially symmetric kernels

For the Gaussian kernel put $r = \lVert x - x' \rVert_2$, the distance between the two points. Then the kernel is a function of the distance alone:

$$K(x, x') = \exp(-\frac{r^2}{2\sigma^2}) = K(r)$$

Every pair of points at the same distance $r$ gets the **same similarity**, so such kernels are called **radially symmetric**. The width $\sigma$ controls how quickly similarity dies off:




Small $\sigma$: only very close neighbours count as similar. Large $\sigma$: even far-away points still look similar.

> **Note for Review — Pro-Tip:** two caveats worth remembering. (1) The Laplacian kernel is written above as in the lecture; the more common textbook form uses the **unsquared** one-norm, $\exp(-\lVert x - x' \rVert_1 / \sigma)$. (2) Polynomial and Gaussian kernels are always PSD, but the **sigmoid kernel is PSD only for some choices of** $a, b$ — so "check PSD before you trust a kernel" is a genuine rule, not a formality. The Gaussian kernel's feature space is in fact infinite-dimensional, which is why we *never* want to construct $\phi$ explicitly for it.

# Kernel Linear Regression

The kernel method applies to **any** machine-learning problem: we just map the data to a higher-dimensional space, and the analysis stays the same. Let's redo **(ridge) linear regression** with kernels.

## Ordinary linear regression first

Data set: $\mathrm{S} = \{(x_1, y_1), \ldots, (x_n, y_n)\}$ with continuous targets $y_i$. The hypothesis is a linear projection:

$$h(x_i) = w^\top x_i = \hat{y}_i \approx y_i$$

We want $\hat{y}_i$ as close as possible to the ground truth $y_i$. To avoid over-fitting we add a regularisation term on $w$:

$$\ell(w) = \frac{1}{2}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 + \frac{\lambda}{2} w^\top w = \frac{1}{2}\sum_{i=1}^{n}(w^\top x_i - y_i)^2 + \frac{\lambda}{2} w^\top w$$

Set the gradient to zero:

$$\nabla \ell(w) = \sum_{i=1}^{n}(w^\top x_i - y_i)x_i + \lambda w = 0 \quad \Longrightarrow \quad w = -\frac{1}{\lambda}\sum_{i=1}^{n}(w^\top x_i - y_i)x_i$$

## Enter the kernel: replace $x_i$ by $\phi(x_i)$

$$h(\phi(x_i)) = w^\top \phi(x_i), \qquad w = -\frac{1}{\lambda}\sum_{i=1}^{n}(w^\top \phi(x_i) - y_i)\,\phi(x_i)$$

Absorb the scalar $-\frac{1}{\lambda}(w^\top \phi(x_i) - y_i)$ into a single number $a_i$:

$$w = \sum_{i=1}^{n} a_i\, \phi(x_i) = \Phi^\top a$$

where $\Phi \in \mathrm{R}^{n \times m}$ has $\phi(x_i)^\top$ as its $i$-th row (first row $\phi(x_1)^\top$, second row $\phi(x_2)^\top$, ...) and $a = (a_1, \ldots, a_n)^\top$. **The optimal weight vector is a linear combination of the lifted data points.**

## Rewrite the loss in terms of $a$

Substitute $w = \Phi^\top a$, so $w^\top \phi(x_i) = a^\top \Phi\, \phi(x_i)$ and $w^\top w = a^\top \Phi\Phi^\top a$:

$$J(a) = \frac{1}{2}\sum_{i=1}^{n}(a^\top \Phi\, \phi(x_i) - y_i)^2 + \frac{\lambda}{2} a^\top \Phi\Phi^\top a$$

Collect the $n$ predictions into a vector, $\hat{\mathbf{y}} = \Phi\Phi^\top a$, and write the sum of squares as a norm, $\frac{1}{2}(\Phi\Phi^\top a - \mathbf{y})^\top(\Phi\Phi^\top a - \mathbf{y})$, where $\mathbf{y}$ stacks all target values. Expanding:

$$J(a) = \frac{1}{2} a^\top \Phi\Phi^\top \Phi\Phi^\top a \;-\; a^\top \Phi\Phi^\top \mathbf{y} \;+\; \frac{1}{2}\mathbf{y}^\top \mathbf{y} \;+\; \frac{\lambda}{2} a^\top \Phi\Phi^\top a$$

Now notice that the $(i,j)$ entry of $\Phi\Phi^\top$ is $\phi(x_i)^\top \phi(x_j) = K(x_i, x_j)$ — so $\Phi\Phi^\top$ **is the kernel matrix** $K$ (in the proof above this is $\Theta\Theta^\top$). Therefore

$$\boxed{J(a) = \frac{1}{2} a^\top K K\, a \;-\; a^\top K\, \mathbf{y} \;+\; \frac{1}{2}\mathbf{y}^\top \mathbf{y} \;+\; \frac{\lambda}{2} a^\top K\, a}$$

The loss now depends on the data **only through** $K$.

## Minimise over $a$

Using $K = K^\top$ (so $\nabla_a\, a^\top M a = 2Ma$ for symmetric $M$, and $\nabla_a\, a^\top K\mathbf{y} = K\mathbf{y}$):

$$\nabla_a J = \frac{2}{2} KKa \;-\; K^\top \mathbf{y} \;+\; \frac{2\lambda}{2} Ka = 0$$

$$KKa + \lambda Ka = K^\top \mathbf{y} = K\mathbf{y}$$

Factor $K$ out of the left side:

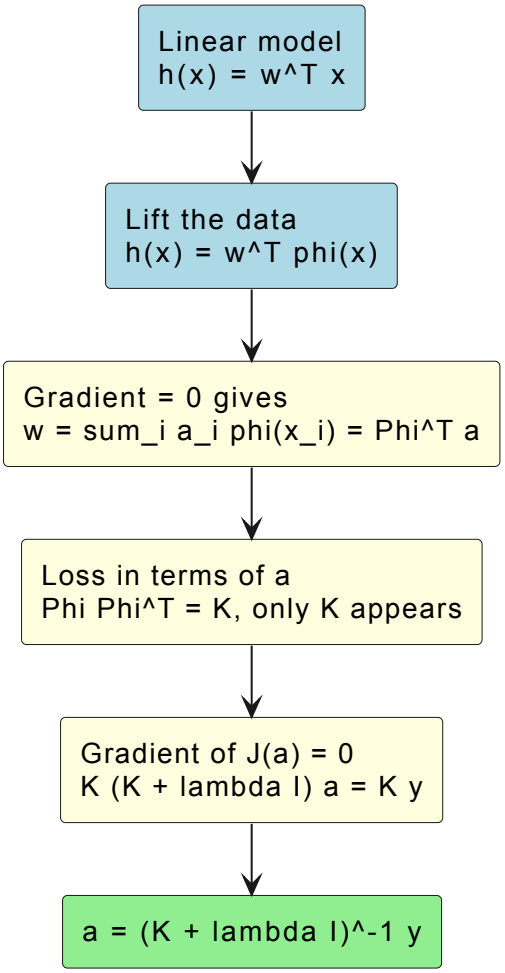
$$K\,(K + \lambda I)\,a = K\,\mathbf{y}$$

Since $K$ is invertible, cancel it:

$$(K + \lambda I)\,a = \mathbf{y} \quad \Longrightarrow \quad \boxed{a = (K + \lambda I)^{-1}\,\mathbf{y}}$$

This is **kernel linear regression**: the parameters follow from the target vector and the kernel matrix alone. To predict at a new point $x$:

$$h(x) = w^\top \phi(x) = \sum_{i=1}^{n} a_i\, \phi(x_i)^\top \phi(x) = \sum_{i=1}^{n} a_i\, K(x_i, x)$$




> **Note for Review — Pro-Tip:** "Is $K + \lambda I$ invertible?" is a classic question. The eigenvalues of a PSD matrix $K$ are $\geq 0$, so those of $K + \lambda I$ are $\geq \lambda > 0$ — it is **always invertible when** $\lambda > 0$, even when $K$ itself is singular. The regulariser is doing double duty: fighting over-fitting *and* keeping the algebra well-posed.

**Takeaway of the whole module.** Whenever we want classes to become linearly separable — or a fitting function to be linear in a lifted space although it is non-linear in the original — we use kernels: similarities in the high-dimensional space come directly from the original points, no mapping needed. If the mapping is needed, build the kernel matrix, eigen-decompose it, and reconstruct $\theta$ as in the Mercer proof. Next up: neural networks.

## Practice Questions

1. **The suspicious similarity table.** A colleague proposes, for two data points, the kernel matrix $K = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}$ ("a point is more similar to the other point than to itself!"). Find a vector $c$ that proves it is *not* a valid kernel matrix, then explain — using what you know about the eigenvalues and about $\theta(x_i)^\top \theta(x_j)$ — why no real feature vectors could ever produce it.

2. **Counting dimensions.** For inputs in $\mathrm{R}^3$, the kernel $(x^\top x' + c)^2$ corresponds to an explicit $\phi$. How many coordinates does $\phi$ have? Now imagine inputs in $\mathrm{R}^{100}$ and a power of $6$. Compare, in rough terms, the work of evaluating the kernel directly with the work of building $\phi$ and taking a dot product. What does this say about *why* the trick is worth the trouble — and what happens in the Gaussian case, where $\phi$ is infinite-dimensional?

3. **Dialing $\sigma$.** Using a Gaussian kernel on $n$ distinct training points, think about the two extremes $\sigma \to 0$ and $\sigma \to \infty$. What does the kernel matrix $K$ approach in each case? What do the eigenvalues and feature vectors $\theta(x_i)$ look like? Combine this with $a = (K + \lambda I)^{-1}\mathbf{y}$ and $h(x) = \sum_i a_i K(x_i, x)$ to predict how the fitted function behaves on the training points and *between* them in each extreme.

4. **Duplicate points, different labels.** Suppose two training points are identical ($x_1 = x_2$) but have different targets $y_1 \neq y_2$. What does this do to the kernel matrix $K$ (rank, invertibility)? Why does the step "cancel $K$ from $K(K + \lambda I)a = K\mathbf{y}$" become dangerous, and what does a small $\lambda > 0$ end up doing to the predicted values at that point? Argue from eigenvalues, not from the final formula.

5. **Kernels can be combined.** Let $K_1$ and $K_2$ be PSD kernel matrices on the same points. (a) Using only the definition $c^\top K c \geq 0$, show that $K_1 + K_2$ is also PSD. (b) If $\theta_1(x)$ and $\theta_2(x)$ are feature maps for $K_1$ and $K_2$, construct a feature map for $K_1 + K_2$ and verify it reproduces every entry. (c) Is "negative squared distance", $K(x, x') = -\|x - x'\|^2$, a valid kernel? Test it on two distinct points before answering.